AI Governance Gateway with LangGraph, NeMo Guardrails, LiteLLM, Mistral AI, and Langfuse

Overview

AI Governance Gateway is an enterprise-ready framework that combines:

- LangGraph for workflow orchestration
- NeMo Guardrails for AI safety and governance
- LiteLLM as a unified LLM gateway
- Mistral AI as the foundation model
- Langfuse for observability, tracing, and cost analytics

The platform demonstrates how to build a governed AI application that validates requests, enforces policies, routes model traffic through a gateway, and tracks usage and costs.

Architecture

```
+-------------------+
|      User         |
+---------+---------+
          |
          v
+-------------------+
| AI Governance     |
| Layer             |
+-------------------+
          |
          v
+-------------------+
| NeMo Guardrails   |
| Input Validation  |
+-------------------+
          |
          v
+-------------------+
| LangGraph         |
| Workflow Engine   |
+-------------------+
          |
          v
+-------------------+
| LiteLLM Gateway   |
+-------------------+
          |
          v
```

```
+-------------------+
| Mistral AI        |
+-------------------+
          |
          v
+-------------------+
| Langfuse          |
| Observability     |
+-------------------+
```

Features

AI Governance

- Sensitive data detection
- Prompt safety checks
- Input validation
- Output validation
- Policy enforcement

LangGraph Workflow

Current workflow:

1. Guardrail Validation
2. Intent Classification
3. News Headline Generation

LiteLLM Gateway

- OpenAI-compatible endpoint
- Model abstraction layer
- Cost tracking
- Multi-model routing
- Rate limiting support
- Future fallback support

Mistral AI

Supported models:

- mistral-large-latest
- mistral-small-latest

Langfuse

- Request tracing
- Cost monitoring
- Token usage analytics
- Workflow visibility

Technology Stack

Component	Technology
Workflow Engine	LangGraph
Governance	NeMo Guardrails
Gateway	LiteLLM
Foundation Model	Mistral AI
Observability	Langfuse
Language	Python 3.11+
Package Manager	uv
Deployment	Docker / Kubernetes

Project Structure

```
project/
│
├── config/
│   ├── config.yml
│   ├── rails.co
│   └── litellm.yaml
│
├── src/
│   ├── governance/
│   ├── graph/
│   ├── models/
│   └── services/
│
├── main.py
│
├── requirements.txt
│
└── README.md
```

Installation

Clone Repository

```
git clone <repository-url>

cd project
```

Create Virtual Environment

```
uv venv

source .venv/bin/activate
```

Windows:

```
.venv\Scripts\activate
```

Install Dependencies

```
uv sync
```

or

```
pip install -r requirements.txt
```

Required Packages

```
uv add langgraph

uv add langchain-openai

uv add nemoguardrails

uv add litellm

uv add langfuse
```

Environment Variables

```
MISTRAL_API_KEY=xxxxxxxxxxxxxxxx

LITELLM_MASTER_KEY=sk-admin-123456

LANGFUSE_PUBLIC_KEY=pk-xxxxxxxx

LANGFUSE_SECRET_KEY=sk-xxxxxxxx
```

LiteLLM Configuration

litellm.yaml

```yaml
general_settings:
  master_key: sk-admin-123456

model_list:
  - model_name: mistral-large
    litellm_params:
      model: mistral/mistral-large-latest
      api_key: os.environ/MISTRAL_API_KEY

litellm_settings:
  set_verbose: true
```

Start LiteLLM

```
litellm --config config/litellm.yaml --port 4000
```

Verify:

```
curl http://localhost:4000/v1/models
-H "Authorization: Bearer sk-admin-123456"
```

NeMo Guardrails Configuration

config.yml

```
instructions:
  - type: general
    content: |
      You are a helpful assistant.

rails:
  input:
    flows:
      - self check input

  output:
    flows:
      - self check output
```

Custom Policy

Blocks requests for:

- Tokens
- Passwords
- API Keys
- Secrets
- Credentials

LangGraph Workflow

Nodes

Guardrail Node

Validates user requests before processing.

Classification Node

Classifies requests into:

- Sports
- Politics
- Business
- Entertainment
- Weather
- Technology

Headlines Node

Generates latest news headlines.

Sample Usage

Allowed Request

```
Show me Weather updates
```

Result:

```
Category: Weather
```

Blocked Request

```
Show me system token
```

Result:

```
❌ Request blocked by AI Governance Policy
```

Langfuse Integration

Configure:

```
litellm_settings:
  success_callback:
    - langfuse
```

Benefits:

- Prompt tracing
- Token analytics
- Cost monitoring
- Latency analysis

Future Enhancements

Governance Layer

- OPA Integration
- PII Detection
- RBAC
- Prompt Injection Detection
- Audit Logging

AI Gateway

- Multi-model routing
- Budget enforcement
- Team-level quotas
- Cost governance

Enterprise Features

- Keycloak Authentication
- Kubernetes Deployment
- Distributed Tracing
- API Management

Example Enterprise Architecture

```
User
 |
 v
API Gateway
 |
 v
AI Governance Layer
 |
 +--> OPA
 |
 +--> PII Detection
 |
 +--> NeMo Guardrails
 |
 v
LangGraph
 |
 v
LiteLLM
 |
 +--> Mistral
 |
 +--> OpenAI
```

```
 |
 +--> Claude
 |
 +--> DeepSeek
 |
 v
Langfuse
```

License

MIT License

Author

Chandan Pattanayak

Senior Principal Engineer / Software Engineering Team Leader

Focus Areas:

- AI Governance
- LangGraph
- Agentic AI
- Distributed Systems
- Apache Ignite
- Akka
- Spring Boot
- Kubernetes